

Complete Architecture & Optimization Documentation

Medical Document Parser - RAG Pipeline

OCR | Chunking | Embedding | AI Search | Extraction | Evaluation | Cost

Current: **\$8,800 / 100 providers | ~2 hours**

Target: **\$750 - \$1,000 / 100 providers | ~20 min**

@VK_Venkatkumar | EXL Service | May 2026

1. End-to-End Pipeline Overview

The pipeline processes documents through 8 stages. Each stage has specific Azure services, token consumption, and optimization opportunities.

Stage	What Happens	Azure Service	Input	Output	Token Cost	Time/Doc	% Total Cost
1. OCR	Extract text from PDF/image	Document Intelligence	Document bytes	Full text + page count	\$0.01/page	3-5 sec	1%
2. Image Convert	PDF to page images (multimodal)	Local (PyMuPDF)	PDF bytes	Base64 images	Free	2-3 sec	0%
3. Chunking	Split text into sections	Local (Python)	Full OCR text	5-8 chunks (3000 chars each)	Free	<1 sec	0%
4. Embedding	Convert chunks + queries to vectors	Azure OpenAI Embedding	Text chunks	1536-dim vectors	\$0.10/1M tokens	1-2 sec	<1%
5. Indexing	Store vectors in search index	Azure AI Search	Vectors + metadata	Indexed documents	\$0.01/1K queries	<1 sec	<1%
6. RAG Search	Find relevant chunks per field	Azure AI Search	Query vector	Top 3 chunks	Included above	<1 sec	<1%
7. GPT Extract	Extract field value from chunks	Azure OpenAI GPT-5	Prompt + chunks + image	Value + confidence	~5000 tokens/call	13 sec	97%
8. Evaluation	Score extraction quality	Azure OpenAI GPT-5	Value + context	6 metric scores	~500 tokens/call	2-3 sec	2%

Key insight: Stage 7 (GPT Extraction) accounts for 97% of total cost. All optimization must target this stage.

2. OCR Stage - Document Intelligence

Azure Document Intelligence extracts text from all document formats. This is the starting point for all extraction.

Aspect	Current	Details
Service	Azure Document Intelligence v4.0	Prebuilt-read model, handles handwritten + printed text
Supported Formats	PDF, PNG, JPG, TIFF, BMP, DOCX, RTF	7 formats supported
Output	Full text + page count + word coordinates	Preserves layout and reading order
Cost	\$0.01 per page	Fixed per page, not per token
Typical Usage	87 pages per provider (20 docs)	\$0.87 per provider
Latency	3-5 seconds per document	Depends on page count and complexity
OCR Text Storage	Stored in memory for hallucination check	Passed to evaluation engine, never saved to disk
Optimization	Cannot reduce - need all text	Only optimization: skip duplicate/blank documents

3. Semantic Chunking Stage

OCR text is split into overlapping chunks that respect document section boundaries. This is critical for RAG quality.

Parameter	Current Value	Purpose	Impact if Changed	Optimization
Target Chunk Size	3000 chars	Balance between context and token cost	Smaller = more chunks, more searches Larger = fewer chunks, less granular	Test 5000 chars to reduce chunk count
Min Chunk Size	500 chars	Prevent tiny useless chunks	Smaller = more noise Larger = may lose small sections	Keep at 500
Max Chunk Size	6000 chars	Prevent oversized chunks	Larger = more tokens per prompt	Keep at 6000
Overlap	300 chars	Ensure context at boundaries	Less = may miss info at boundaries More = duplicate content	Keep at 300
Strategy	Section-aware adaptive	Respects headers, numbered sections	Fall back to overlap-based if no sections found	No change needed

Chunking Impact on Cost:

Chunk Size	Chunks/Doc (avg)	Embedding Calls	Search Calls	GPT Prompt Size	Impact
2000 chars	10-15	More (+50%)	Same	Smaller (-20%)	More granular but more embedding cost
3000 chars (current)	5-8	Baseline	Baseline	Baseline	Good balance
5000 chars	3-5	Fewer (-40%)	Same	Larger (+30%)	Fewer chunks but larger prompts
Entire document (no chunking)	1	Minimal	None needed	Very large	May exceed context window

4. Embedding Stage

Text chunks and field queries are converted to vectors for similarity search. Embeddings enable semantic matching between field descriptions and document content.

Aspect	Current	Details	Optimization
Model	text-embedding-ada-002 / text-embedding-3-small	3584 dimensions	Smaller model (256-dim) saves storage but slightly lower quality
Chunk Embeddings	5-8 per document (once)	Embedded during indexing	No optimization needed - runs once
Query Embeddings	34 per document (one per field)	Same query repeated across all docs	CACHE: Same 34 queries every run. Cache embeddings, reuse across docs. Saves 95% of query embeddings
Token Cost	\$0.10 per 1M tokens	~100 tokens per embedding	Negligible cost (\$0.01 per provider)
Total Calls/Provider	~748 (34 queries x 20 docs + 8 chunks x 20 docs)	Most are duplicate query embeddings	With cache: 34 + 160 = 194 calls (74% reduction)

Embedding Cache Implementation:

Current: For each document, generate 34 field query embeddings (identical every time). 20 docs x 34 = 680 embedding calls per provider. All 680 produce the same 34 vectors.

With Cache: Generate 34 embeddings once at start. Store in memory dict. Reuse for all 20 documents. Total: 34 embedding calls instead of 680. Saves 646 calls per provider.

Cost saving: minimal (\$0.006/provider). Speed saving: ~10% faster. Effort: 1 day.

5. Azure AI Search Stage

Vector index stores embedded chunks with metadata. Per-field RAG search finds the most relevant chunks for each extraction query.

Aspect	Current	Details	Optimization
Index Type	Universal vector index	Single index per doctype	No change needed
Vector Dimensions	1536	Matches embedding model	Could reduce with smaller embedding model
Metadata Fields	provider, folder, upload_timestamp	OData filtering per search	No change needed
Search Type	Vector search (cosine similarity)	top_k: 3 chunks returned	Reduce to top_k: 2 saves ~30% prompt tokens
Reranking	Optional (config: false)	Semantic reranker after vector search	Disabled - minimal benefit for per-field search
Auto-Cleanup	Delete chunks older than N days	Prevents index bloat	Set to 2-7 days based on reprocessing needs
Search Calls/Provider	680 (34 fields x 20 docs)	Each field gets its own search	With document classification: ~160 calls
Cost	\$0.01 per 1000 queries	\$0.007 per provider	Negligible

Search Quality vs Cost Trade-off:

top_k	Chunks Returned	Prompt Size	Token Cost	Accuracy	Recommendation
1	1 chunk	~1500 tokens	Lowest	May miss context	Not recommended
2	2 chunks	~3000 tokens	-30% vs current	~2% drop	Test for cost reduction
3 (current)	3 chunks	~4500 tokens	Baseline	Best	Current production setting
5	5 chunks	~7500 tokens	+50% vs current	Marginal improvement	Not recommended - diminishing returns

6. Field Extraction Stage (97% of Cost)

This is the most expensive stage. GPT-5 extracts each field value from RAG chunks with optional document image (multimodal). Per-field RAG ensures focused extraction.

6.1 Current Architecture: Field-Centric

Aspect	Current	Impact
Approach	One GPT call per field per document	34 calls per document = highest accuracy, highest cost
Prompt Size	~5000 tokens (system + field def + 3 chunks + image)	\$0.05 per call (input) + \$0.03 per call (output)
Multimodal	Document image included in every call	Image adds ~2000-3000 tokens per call
Parallel Workers	15 concurrent field extractions	Speed improvement only, same total cost
Total Calls/Provider	680 (34 fields x 20 docs)	\$85 per provider
Empty Results	~70% of calls return empty	~\$60 wasted per provider on fields that dont exist in that document
RAG Search	Dedicated search per field	Best chunk selection per field
Prompt Builder	Universal (reads any key from field_info)	Auto-includes notes, special_handling, validation

6.2 Three Alternative Architectures

Architecture	How It Works	GPT Calls/Doc	GPT Calls/Provider	Cost/Provider	Accuracy	Dev Effort
Field-Centric (current)	For each field: search + extract (1 call per field)	34	680	\$88	Best	-
Document-Centric	For each doc: send all text + all 34 fields. 1 call extracts everything	20	20	\$3	~10-15% drop	1 week
Chunk-Centric	For each chunk: extract ALL possible fields from that chunk	5-8	100-160	\$12-18	~3-5% drop	2 weeks
Hybrid (Classification + Chunk)	Classify doc type first. Then extract only relevant fields per chunk	4-6	80-120	\$8-12	~2-3% drop	2 weeks

6.3 Document Classification Architecture (Recommended #1)

Classify each document type first using a cheap model call. Then extract only the fields relevant to that document type. This eliminates 70% of wasted GPT calls.

Document Type	Fields to Extract	Count	Instead of 34	Calls Saved/Doc
License (State/Medical)	license_number, license_state, issue_date, expiration_date, status, first_name, last_name, npi	7	26 skipped	26
DEA Certificate	dea_number, dea_state, dea_issue_date, dea_expiration_date	4	30 skipped	30
Board Certification	board_name, board_specialty	2	32 skipped	32
BLS/ACLS Card	bls_issue_date, bls_expiration_date	2	32 skipped	32
Vaccine Record	tb, flu, covid, hepB, mmr, varicella, tdap vaccines	7	27 skipped	27
Insurance Form	insurance_name, address, city, state, postal, email, policy, expiration	8	26 skipped	26
CV/Resume/Application	first_name, last_name, email, cell, birth_date, ssn, degree, gender, npi, drivers_license	10	24 skipped	24
Other (unknown)	ALL 34 fields (fallback - same as current)	34	0 skipped	0

Classification Cost: 1 GPT-4o-mini call per document (~100 tokens = \$0.000015). Virtually free.

Savings: 20 docs x ~26 saved calls x \$1.30/call = \$676 saved per provider out of \$884 total = 76% reduction.

6.4 Chunk-Centric Architecture (Recommended #2)

Instead of searching for each field separately, read each chunk once and extract ALL possible fields from it.

Step	Field-Centric (Current)	Chunk-Centric (Proposed)
Input	1 field query	1 document chunk
Search	Vector search for this specific field	No search needed (already have the chunk)
Prompt	"Extract license_number from these chunks"	"Extract ALL fields you can find in this section"
GPT Output	{"license_number": {"value": "56359"}}	{"license_number": "56359", "license_state": "KY", "expiration": "12/2025"}
Calls per doc	34 (one per field)	5-8 (one per chunk)
Duplicate reading	YES - same chunk read 3-4 times for related fields	NO - each chunk read exactly once
Empty results	~70% calls return empty	~5% (only if chunk has no extractable data)
Cost per doc	34 x \$0.08 = \$2.72	6 x \$0.10 = \$0.60

Why chunk-centric has slightly lower accuracy: GPT processes more fields at once. For complex multi-value fields (licenses with dates), focused per-field prompts are better. Solution: Hybrid approach uses classification to limit fields, then chunk-centric for extraction.

7. Evaluation Stage

6 evaluation metrics assess extraction quality. 4 use GPT calls, 2 are free local checks.

Metric	Type	What It Checks	Input Needed	Cost	Can Optimize?
LLM-as-Judge	GPT call	Format, normalization, quality per field	Value + guidelines	~500 tokens/field	Run on sample (10%)
Hallucination	Local (free)	Value exists in source OCR text?	Value + OCR text	Free	No optimization needed
Completeness	Local (free)	% of fields with values	All field values	Free	No optimization needed
Groundedness	GPT call	Value supported by RAG chunks?	Value + RAG chunks	~800 tokens/field	Run on sample (10%)
Relevance	GPT call	Value relevant to field query?	Value + query	~300 tokens/field	Run on sample (10%)
Retrieval	GPT call	RAG search returned right chunks?	Query + chunks	~800 tokens/field	Run on sample (10%)

Evaluation Cost Optimization:

Strategy	Cost Impact	Quality Impact	How
Run eval on ALL providers (current)	Baseline (~\$2/provider)	Every field evaluated	evaluation.enabled: true
Run eval on 10% sample	-90% eval cost	Sample quality only	Enable for every 10th provider
Disable eval for bulk runs	-100% eval cost	No quality scores	evaluation.enabled: false
Free metrics only (hallucination + completeness)	-80% eval cost	Basic quality checks	Disable LLM judge + azure_ai_eval

8. Complete Cost Comparison - All Architectures

Component	Current Field-Centric	Config Optimized	Classification Only	Chunk-Centric Only	Hybrid (Classify+Chunk)	Full Optimized
OCR calls	20 docs	20 docs	20 docs	20 docs	20 docs	15 docs (skip dups)
OCR cost	\$0.87	\$0.87	\$0.87	\$0.87	\$0.87	\$0.65
Classification calls	0	0	20 (cheap)	0	20 (cheap)	15 (cheap)
Classification cost	\$0	\$0	\$0.003	\$0	\$0.003	\$0.002
Embedding calls	748	748	194 (cached)	160 (chunks only)	194 (cached)	149 (cached)
Embedding cost	\$0.01	\$0.01	\$0.003	\$0.002	\$0.003	\$0.002
Search calls	680	680	160	0 (no search)	0 (no search)	0
Search cost	\$0.007	\$0.007	\$0.002	\$0	\$0	\$0
GPT extraction calls	680	680	160	120	80-100	60-80
GPT extraction tokens	3,400,000	3,400,000	800,000	720,000	480,000	360,000
GPT extraction cost	\$85.00	\$85.00	\$20.00	\$18.00	\$12.00	\$9.00
Evaluation cost	\$2.00	\$0 (disabled)	\$0.50 (sample)	\$0.50	\$0.50	\$0.25
TOTAL / provider	\$88.00	\$86.00	\$21.50	\$19.50	\$13.50	\$10.00
TOTAL / 100 providers	\$8,800	\$8,600	\$2,150	\$1,950	\$1,350	\$1,000
Annual (48 runs)	\$422,400	\$412,800	\$103,200	\$93,600	\$64,800	\$48,000
Annual savings	-	\$9,600	\$319,200	\$328,800	\$357,600	\$374,400

9. Performance (Speed) Comparison

Metric	Current	Config Only	Classification	Chunk-Centric	Hybrid	Full Optimized
GPT calls/doc	34	34	8	5-8	4-6	4-5
GPT calls/provider	680	680	160	120	80-100	60-80
Time per GPT call	13 sec	13 sec	13 sec	13 sec	13 sec	13 sec
Parallel workers	15	20	15	15	15	20
Time/provider (extraction)	10 min	7.5 min	2.3 min	1.7 min	1.2 min	0.7 min
Time/provider (total)	15 min	12 min	5 min	4 min	3 min	2 min
Time/100 providers	~2 hours	~1.5 hours	~40 min	~30 min	~25 min	~15 min
Speed improvement	-	1.3x	3x	4x	5x	8x

10. Token Usage Comparison

Token breakdown per provider (20 documents, 34 fields):

Token Type	Current	Classification	Chunk-Centric	Hybrid	Full Optimized
Embedding tokens	74,800	19,400	16,000	19,400	14,900
GPT input tokens	2,720,000	640,000	576,000	384,000	288,000
GPT output tokens	680,000	160,000	144,000	96,000	72,000
GPT total tokens	3,400,000	800,000	720,000	480,000	360,000
Eval tokens	68,000	17,000	17,000	17,000	8,500
TOTAL tokens	3,542,800	836,400	753,000	516,400	383,400
Token reduction	-	76%	79%	85%	89%

11. Implementation Roadmap

Phase	What	Cost/100	Savings	Time	Tokens/Provider	Dev Effort
Current	Field-Centric, multimodal, 15 workers	\$8,800	-	~2 hours	3,542,800	-
Phase 1 (Day 1)	Config: workers:20, TPM:5M, content filter, eval off	\$8,600	2%	~1.5 hours	3,474,800	5 min (config)
Phase 2 (Week 1-2)	Document Classification + Embedding cache	\$2,100	76%	~40 min	836,400	1 week
Phase 3 (Week 3-4)	Chunk-Centric extraction for classified documents	\$1,350	85%	~25 min	516,400	2 weeks
Phase 4 (Month 2)	Response cache + prompt compression + skip duplicates	\$1,000	89%	~20 min	383,400	+ 1 week

Annual Impact:

Phase	Monthly (4 runs)	Annual	Annual Savings	ROI (vs dev cost)
Current	\$35,200	\$422,400	-	-
After Phase 1	\$34,400	\$412,800	\$9,600	Immediate (0 dev cost)
After Phase 2	\$8,400	\$100,800	\$321,600	\$321K saved / 1 week dev
After Phase 3	\$5,400	\$64,800	\$357,600	\$357K saved / 3 weeks total
After Phase 4	\$4,000	\$48,000	\$374,400	\$374K saved / 4 weeks total

Bottom Line: From \$422,400/year to \$48,000/year = \$374,400 annual savings. Primary driver: Document Classification (76% of savings) + Chunk-Centric extraction (additional 9%). All with zero or minimal accuracy loss. Total development effort: 4 weeks.